

UT01 – Tecnologías de programación sobre clientes web

ACTIVIDAD EVALUABLE (RA1) – PLANTILLA DE ENTREGA

Nombre y Apellidos del alumno/a:

Gonzalo Rodríguez Fernández

Fecha de entrega:

18/09/2026

URL del repositorio GitHub:

<https://github.com/GonzaloRodFer/JavaScriptDAW>

1. Repositorio y Estructura de la Actividad (0,50 pts)

El repositorio de GitHub ha sido configurado bajo la estructura obligatoria **ACTIVIDADES ENTREGABLES / ACT-1** / y compartido con la profesora (aralvarezr@educa.jcyl.es). A continuación se presentan las evidencias de código y funcionamiento de cada ejercicio.

 **Evidencia: Captura del explorador de archivos donde se vean todos los archivos generados.**

```

  v UT1
    v ACT-1
      v Ej0.0
        <> Ej0.0.html
        JS script.js
      v Ej0.1
        <> Ej0.1.html
      v Ej1.1
        <> Ej1.1.html
        JS script.js
      v Ej1.2
        <> Ej1.2.html
        JS script.js
        JS script2.js
      v Ej2
        JS script.js
```

Ejercicio 1.1 – Carga Diferida con 'defer'

- Estructura: Archivo **index.html** con `<script defer src="..."></script>` ubicado en la cabecera `<head>`. El script genera los números del 1 al 100 en el cuerpo del documento mediante `document.body.innerHTML += "<p>"+i+"</p>"`.

Explica aquí qué ocurre si eliminas el atributo defer de la etiqueta `<script>`

Si eliminas el atributo `defer` de la etiqueta `<script>` no ejecuta el 'innerHTML' porque al tener la etiqueta `<script>` en el `<head>` no carga el html completo sino que va línea por línea y detecta el 'innerHTML' como NULL.

Ejercicio 1.2 – Carga Asíncrona con 'async'

- Estructura: Archivo **index.html** con dos scripts asíncronos enlazados en el `<head>` mediante `<script async src="..."></script>`. Uno muestra los números del 1 al 100 y el otro del 900 al 1000.

📷 Explica aquí qué se pretende demostrar y si se ha conseguido con este ejemplo

Se pretende demostrar que si añadimos dos scripts asíncronos enlazados en el `<head>` mediante `async`, estos dos scripts se ejecuten en paralelo y se muestran los números de los dos scripts intercalados. No lo hemos conseguido con este ejemplo ya que en el resultado podemos ver que no se muestran los números intercalados sino que se muestran primero los de un script y luego los del otro script.

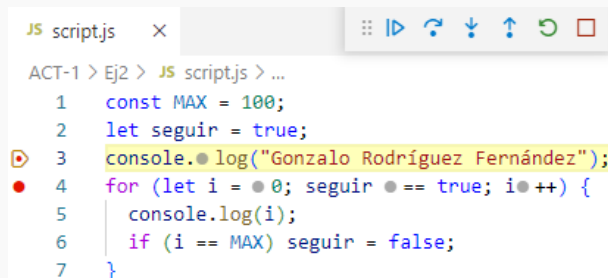
2. Depuración en Visual Studio Code (0,75 pts)

La sesión de depuración se realiza sobre el archivo JavaScript ubicado en la carpeta **Ej.2**. A continuación se documentan los elementos clave de la interfaz de depuración de VS Code.

2.1. Punto de Ruptura (Breakpoint)

📷 Captura de Pantalla: Punto de ruptura (Breakpoint) activo en el código del Ej.2

[Inserta aquí la captura de pantalla legible y añade una breve explicación del resultado mostrado]



```
JS script.js x
ACT-1 > Ej2 > JS script.js > ...
1  const MAX = 100;
2  let seguir = true;
3  console.log("Gonzalo Rodríguez Fernández");
4  for (let i = 0; seguir == true; i++) {
5      console.log(i);
6      if (i == MAX) seguir = false;
7  }
```

En la captura de pantalla vemos 2 puntos de interrupción en el código y como al depurar se para en el primer punto de interrupción.

2.2. Botones de Control de Depuración de VS Code

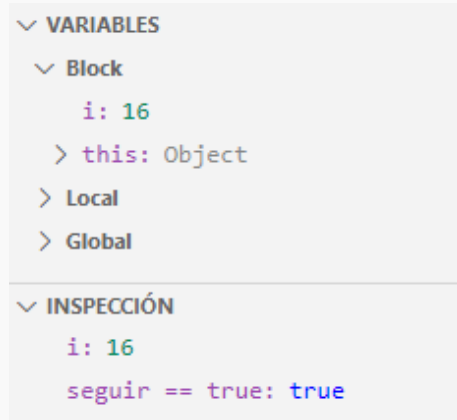
Identificación y explicación técnica del efecto de los 6 botones del panel flotante de depuración:

Botón / Acción	Icono	Atajo de Teclado	Efecto / Descripción Técnica
Continuar (Continue)		F5	Pasa con la lectura y ejecución del siguiente punto de ruptura que hay en el código.
Paso a paso por encima (Step Over)		F10	Ejecuta la siguiente instrucción y si la instrucción contiene una función ejecuta la función sin entrar en ella
Paso a paso por dentro (Step Into)		F11	Ejecuta la siguiente instrucción y si la instrucción contiene una función ejecuta la función entrando en ella
Paso a paso para salir (Step Out)		Máyus+F11	Se ejecuta simplemente la función o método que se está depurando.
Reiniciar (Restart)		Ctrl+Mayús+F5	Reinicia la depuración desde el principio.
Detener (Stop)		Mayús+F5	Detiene la depuración.

2.3. Inspección de Código: Zonas Watch y Variables

 **Captura de Pantalla: Panel 'Watch' con expresiones en seguimiento y 'Variables' en un estado concreto**

[Inserta aquí la captura de pantalla legible y añade una breve explicación del resultado mostrado]



Se muestra el valor que tiene la variable `i` que es '16' y el estado de la variable `seguir` que tiene el estado 'true'.

3. Ejecución con Node.js (0,25 pts)

Ejecución del script de la carpeta Ej2 en el entorno de servidor local Node.js mediante la terminal integrada de Visual Studio Code.

 Captura de Pantalla: Terminal integrada con comando 'node -v', la ruta completa y la ejecución del script mostrando la salida personalizada

[Inserta aquí la captura de pantalla legible y añade una breve explicación del resultado mostrado]

```
PS C:\Users\gonzalo.rodfer.4\JavaScriptDAW\UT1> node -v
v24.21.0
PS C:\Users\gonzalo.rodfer.4\JavaScriptDAW\UT1> cd ./ACT-1/Ej2
PS C:\Users\gonzalo.rodfer.4\JavaScriptDAW\UT1\ACT-1\Ej2> node script.js
Gonzalo Rodríguez Fernández
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
90
91
92
93
94
95
96
97
98
99
100
```

Vemos como node nos ejecuta el script cuando accedemos a la ruta completa donde se encuentra y nos devuelve por consola el resultado que esperabamos, los números del 0 al 100.

4. Lista de Cotejo / Autoevaluación

Calificación máxima: 1,50 pts · Escala: Sí = puntuación total · Parcial = 50 % · No = 0

Criterio de Evaluación	Puntuación Máx.	Sí/Parcial /No
1. Repositorio y estructura	0,50	Sí
· Repositorio compartido con aralvarezr@educa.jcyl.es y accesible (sin 404).	0,05	Sí
· Carpeta ACTIVIDADES ENTREGABLES / ACT-1 con el documento y ejercicios.	0,05	Sí
· Ej.0.0: .html con <script src="..."> y script.js que muestra 'Hola mundo'.	0,10	Sí
· Ej.0.1: .html con script en línea (sin src) y resultado coincidente con Ej.0.0.	0,10	Sí
· Ej.1.1: Script externo con defer en <head> escribe 1-100 en document.body.	0,10	Sí
· Ej.1.2: Dos scripts con async en <head>, uno 1-100 y otro 900-1000.	0,10	Sí
2. Depuración en Visual Studio Code	0,75	Sí
· Puntos de ruptura (breakpoints) con ejecución pausada y línea resaltada.	0,20	Sí
· 6 Botones de avance identificados y explicados con sus efectos.	0,20	Sí
· Zona Watch con expresión añadida y valor durante la ejecución.	0,15	Sí
· Zona Variables comentando cambios de valor en ejecución.	0,15	Sí
· Calidad de evidencia: capturas legibles.	0,05	Sí
3. Ejecución con Node.js	0,25	Sí
· Ejecución del commando en la ruta correcta.	0,10	Sí
· Salida por consola correcta, incluyendo el nombre del alumno	0,10	Sí
· Entorno visible: terminal integrada de VS Code y versión de Node (node -v).	0,05	Sí
CALIFICACIÓN OBTENIDA	1,5	1,5

Observaciones:

[Si quieres añadir alguna observación sobre esta actividad, puedes hacerlo aquí]